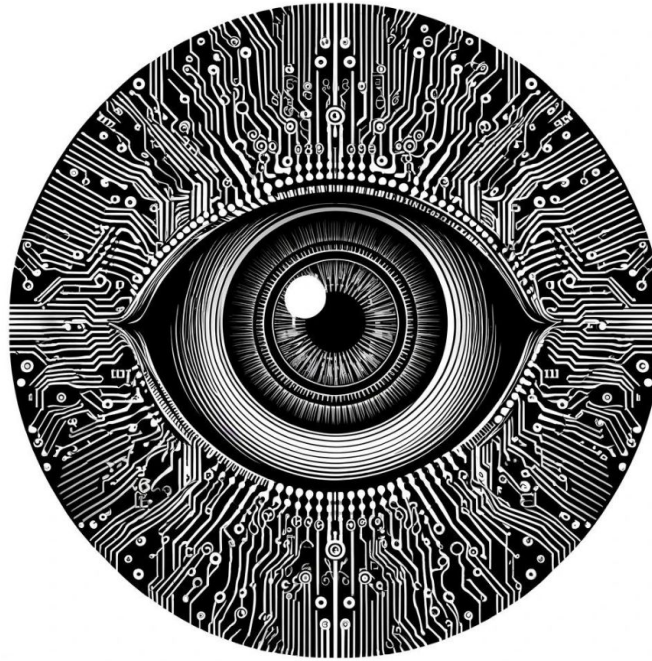


When Images Look Alike



Antonio Rueda-Toicen

About me

- Research Scientist at [Hasso Plattner Institute](#), external DevRel for [Voxel51](#)
- Organizer of the [Berlin Computer Vision Group](#)
- Instructor at [Nvidia's Deep Learning Institute](#) and Berlin's [Data Science Retreat](#)

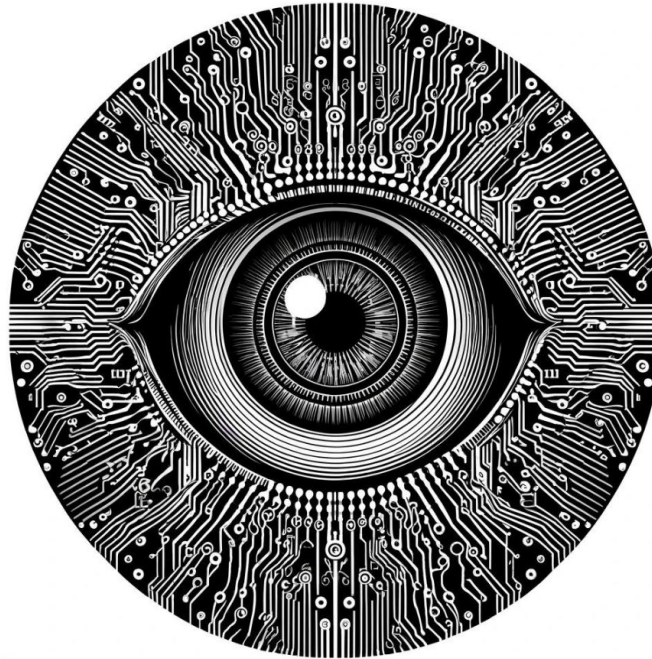


LinkedIn



Slides

Practical Computer Vision with PyTorch at OpenHPI



To be published on
May 7th, 2025 on
OPEN HPI

Antonio Rueda-Toicen

Topics to discuss today

- Image duplicates in big datasets: experience at HomeToGo
- Neural embeddings and perceptual hashing approaches
- Dataset deduplication as a necessary step against data leakage (aka: *cheating*)



hometogo.com

HomeToGo is a search engine for vacation rentals

~15 million properties from ~60000 suppliers

(largest selection worldwide)

Mission: provide best available deals



Booking.com



Problem: matching and merging offers



FERIENWOHNUNG · 5 SCHLAFZIMMER · 22 GÄSTE

ab **332 €**

FerWo-direkt

5,0 ★★★★★

(25 Bewertungen) · Details

ZUM ANGEBOT



FERIENWOHNUNG · 2 SCHLAFZIMMER · 9 GÄSTE

ab **144 €**

FerWo-direkt

4,0 ★★★★★

(9 Bewertungen) · Details

ZUM ANGEBOT



FERIENWOHNUNG · 5 SCHLAFZIMMER · 22 GÄSTE

ab **362 €**

Airbnb

5,0 ★★★★★

Details

ZUM ANGEBOT



FERIENWOHNUNG · 2 SCHLAFZIMMER · 7 GÄSTE

ab **143 €**

FerWo-direkt

4,0 ★★★★★

(7 Bewertungen) · Details

ZUM ANGEBOT



111 M² FERIENHAUS · 2 SCHLAFZIMMER · 8 GÄSTE

ab **193 €**

FerWo-direkt

4,7 ★★★★★

(13 Bewertungen) · Details

ZUM ANGEBOT



FERIENHAUS · 4 SCHLAFZIMMER · 20 GÄSTE

Preis anfragen

Expedia

5,0 ★★★★★

(2 Bewertungen) · Details

ZUM ANGEBOT



3-STAR HOTEL · DOWNTOWN GRAND LAS VEGAS

\$30 ~~\$33~~

Travelocity

4.2 ★★★★★ · Details

VIEW DEAL

Search results in early 2019

Pain with duplicate listings



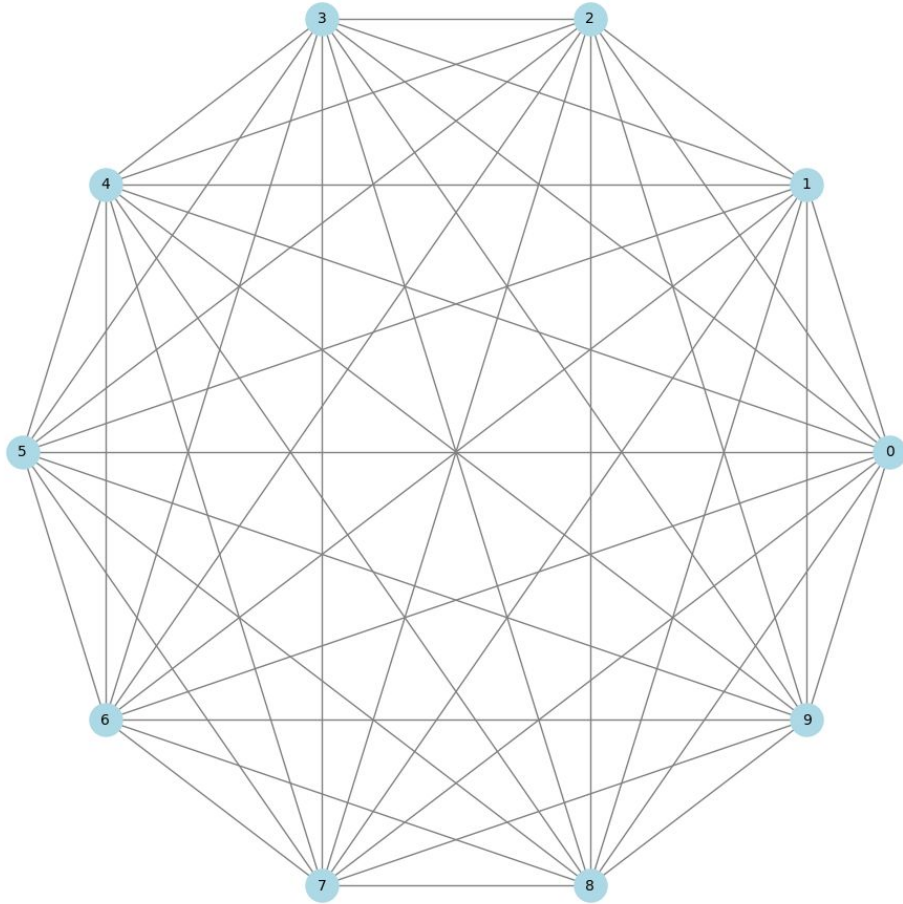
210 EUR per night



232 EUR per night



Humongous number of possible matchings



$$\binom{n}{2} = \frac{n(n-1)}{2}$$

$$\frac{1500 \times 1499}{2} = 1\,124\,250$$

$$1,000,000 \times 15,000,000 = 15,000,000,000,000$$

$$15,000,000,000,000 \times 3 = 45,000,000,000,000$$

Metadata didn't help

```
{
  "provider_a": {
    "property_id": "DBV7842",
    "name": "Sea View Villa Adriatica",
    "owner": {
      "name": "Marina Kovačić",
      "phone": "+385 20 456 789",
      "email": "marina.kovacic@email.hr"
    },
    "location": {
      "address": "Ul. Frana Supila 27",
      "city": "Dubrovnik",
      "coordinates": {
        "latitude": 42.6412,
        "longitude": 18.1146
      }
    },
    "details": {
      "bedrooms": 4,
      "bathrooms": 3,
      "max_guests": 8
    },
    "price_range": "350-750 EUR/night"
  }
}
```

```
{
  "provider_b": {
    "property_id": "DF8123",
    "name": "Seaside Villa in Dubrovnik",
    "location": {
      "area": "Near Old Town",
      "city": "Dubrovnik"
    },
    "details": {
      "bedrooms": 4,
      "bathrooms": 3,
      "max_guests": 8
    },
    "price_range": "350-750 EUR/night"
  }
}
```

Internal competition



Dalle-3 prompt: "illustration of two competing directors inside a startup"

External competition



ACME®

Figure 7: contractor

Hashing as a solution



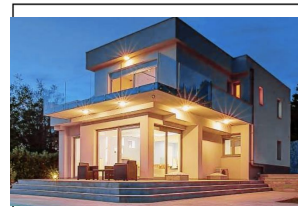
Perceptual hash: **94088af86c038327** (both images)

Cryptographic hashes (md5):

14ee7fe587860078a110903318bd986 (left image)

07aaedb9b75e88a605184f01be5cc50 (right image)

Collisions in hashing



1. Input Image

$h(x)$

2. Hashing Function

4A3F8B2C1D9E5F7A

3. Image Fingerprint

The previous method: DCT hashing

	1	2	3	4	5	6	7	8
1	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE
2	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
3	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE
4	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE
5	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	FALSE	FALSE
6	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
7	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
8	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE

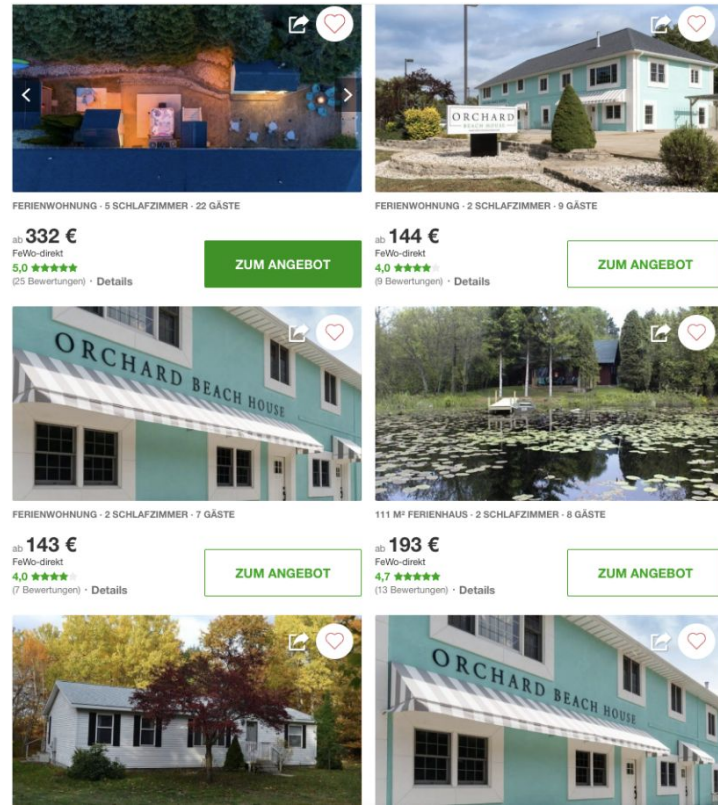
* DCT = Discrete Cosine Transform

$$T_{lf} = \text{median}(Y_{00}, \dots, Y_{77}) + \alpha \cdot \sigma_{lf}$$

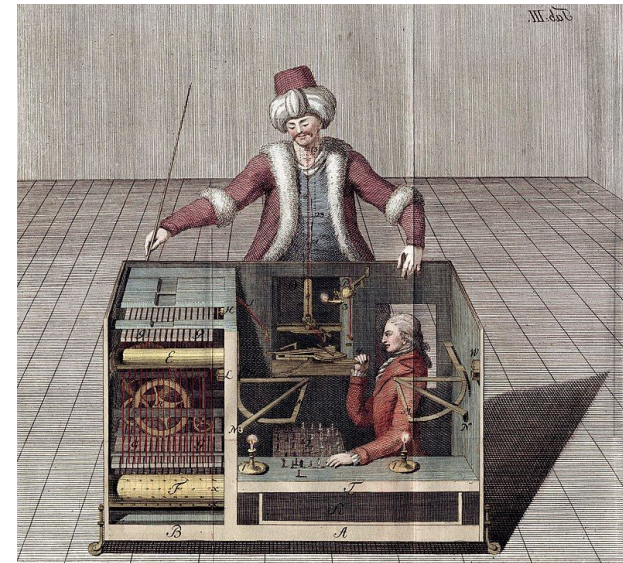
$$b_i = \begin{cases} 1 & \text{if } Y_i \geq T_{lf} \\ 0 & \text{if } Y_i < T_{lf} \end{cases}$$

	1	2	3	4	5	6	7	8
1	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE
2	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
3	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE
4	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE
5	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	FALSE	FALSE
6	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE
7	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE
8	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	TRUE	TRUE

Benchmarks after creating a test set

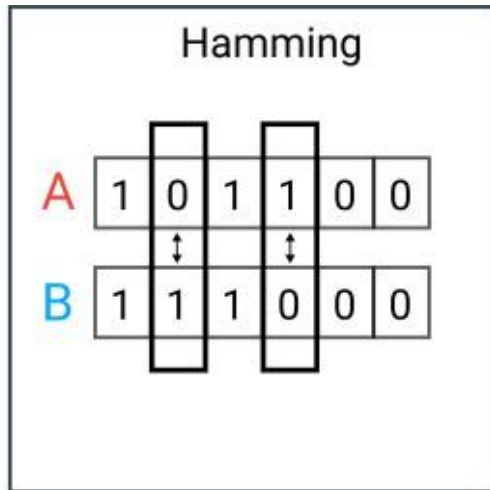


<https://www.mturk.com/>



Transformation Type	Precision	Recall
Crop	0.669	0.665
Crop & Zoom	0.664	0.668
Different Angle	0.452	0.449

Pro: computing Hamming distance is fast



$$d_H(A, B) = \sum_{i=1}^n a_i \oplus b_i = (1 \oplus 1) + (0 \oplus 0) + (1 \oplus 0) + (1 \oplus 1) = 0 + 0 + 1 + 0 = 1$$

Hex	Bin	Hex	Bin	Hex	Bin	Hex	Bin
0	0000	1	0001	2	0010	3	0011
4	0100	5	0101	6	0110	7	0111
8	1000	9	1001	A	1010	B	1011
C	1100	D	1101	E	1110	F	1111

Sparse vs dense vector representations

$$\mathbf{x}_1 = \text{[Image of handwritten 3]} \quad \mathbf{x}_2 = \text{[Image of handwritten 3]} \quad 28 \times 28 \text{ pixels} = 784 \text{ values}$$

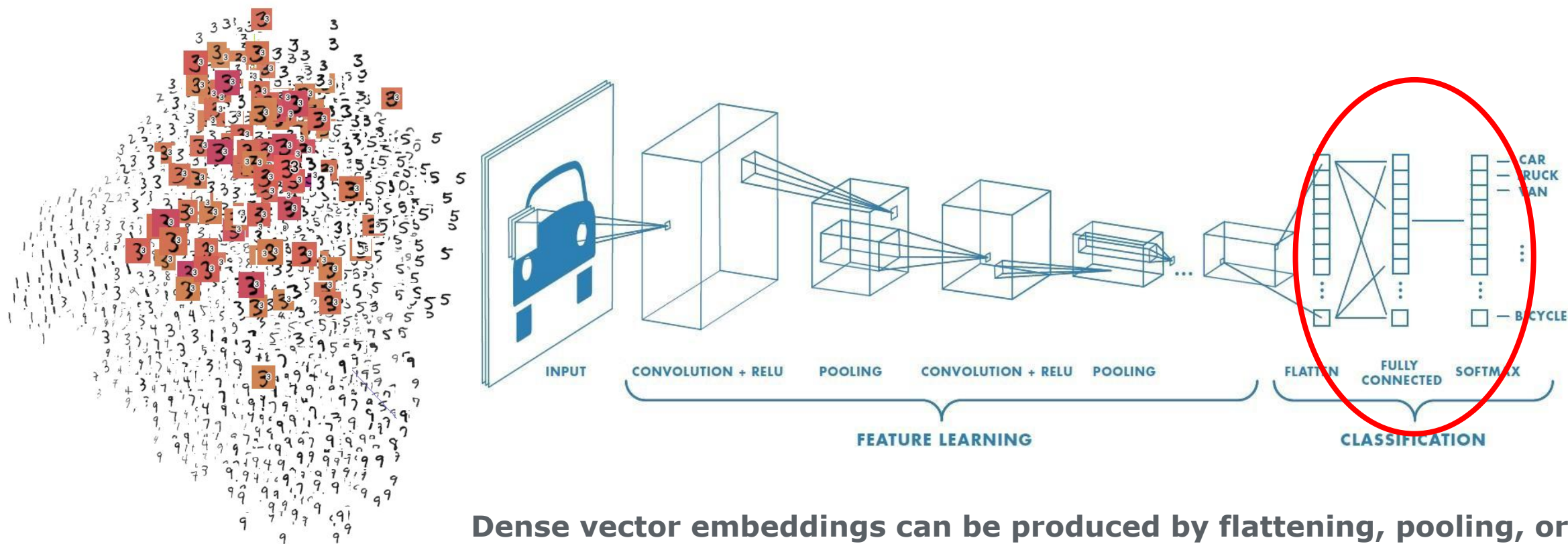
$$\mathbf{x}_1 \text{ sparse} = \mathbf{x}_2 \text{ sparse} = [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^\top$$

$$\mathbf{x}_2 \text{ dense} = [0.10 \quad -0.75 \quad 0.35 \quad 0.41 \quad -0.20 \quad 0.89 \quad -0.60 \quad 0.03 \quad 0.59 \quad -0.50]^\top$$

$$\mathbf{x}_1 \text{ dense} = [0.12 \quad -0.85 \quad 0.37 \quad 0.44 \quad -0.22 \quad 0.90 \quad -0.63 \quad 0.01 \quad 0.58 \quad -0.49]^\top$$

Image embeddings as a byproduct of training

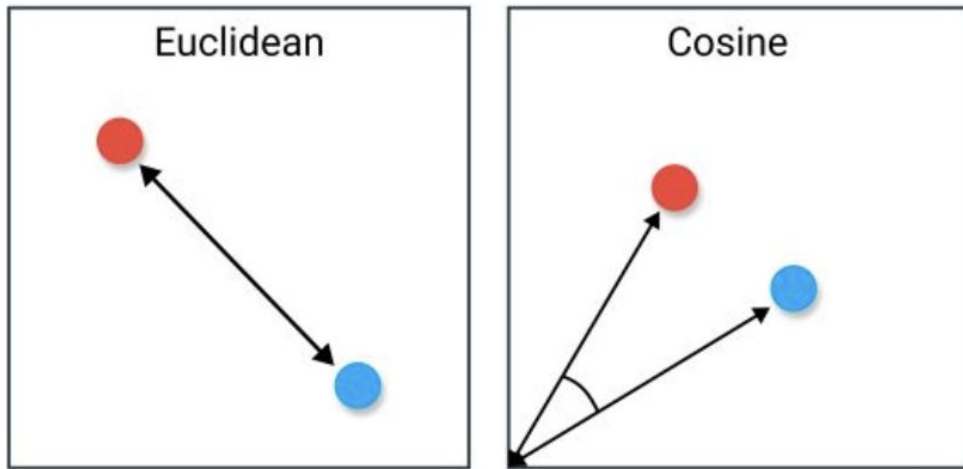
When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other



Dense vector embeddings can be produced by flattening, pooling, or feeding (learned) features to linear layers

[Try exploring this embedding projector](#)

Metrics to compare embeddings



$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

Approaches to produce embeddings with neural networks

```
import torch.nn as nn
```

```
# Approach 1: Direct embedding
```

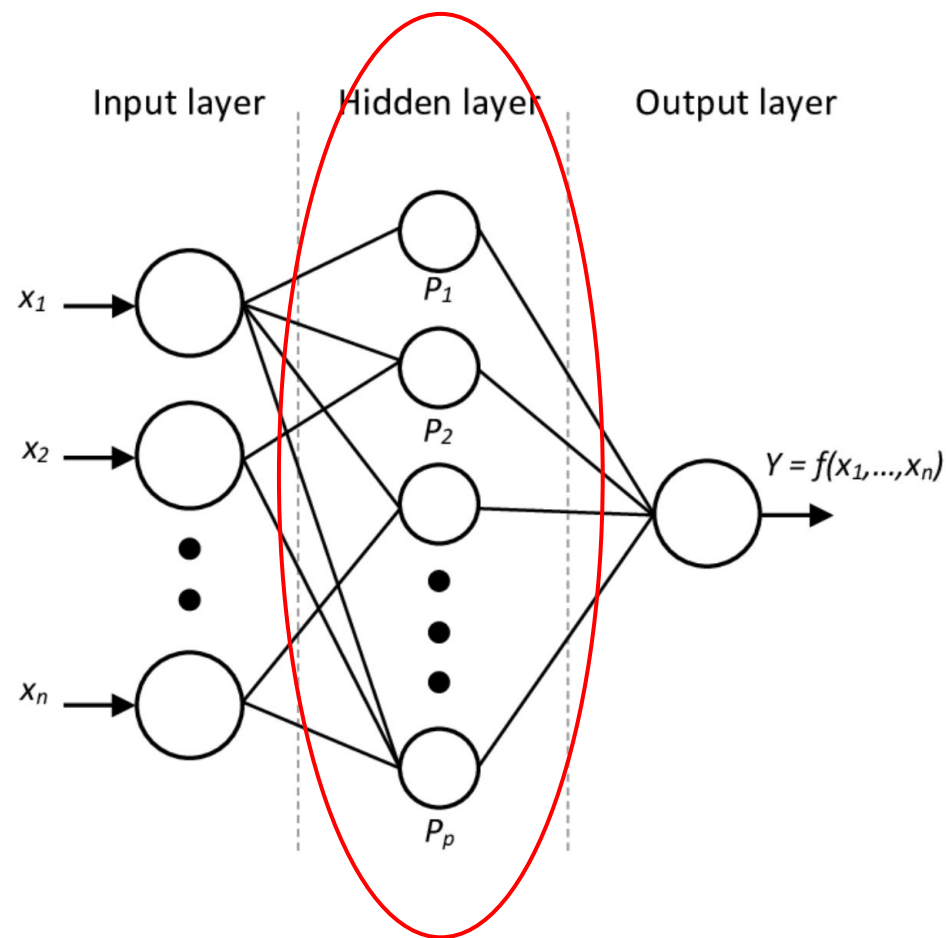
```
linear_embedder = nn.Sequential(  
    nn.Flatten(), # 784  
    nn.Linear(784, 10)  
)
```

```
# Approach 2: Convolution and then fully connected layer
```

```
conv_linear_embedder = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 64, kernel_size=3), # 24x24x64  
    nn.ReLU(),  
    nn.Flatten(), # 24*24*64  
    nn.Linear(24*24*64, 10)  
)
```

```
# Approach 3: Global pooling after convolutions
```

```
conv_pool_embedder = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 10, kernel_size=3), # 24x24x10  
    nn.ReLU(),  
    nn.AdaptiveAvgPool2d((1, 1)), # 1x1x10  
    nn.Flatten() # 10  
)
```



$$L_{L1} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Extract embeddings from a pretrained Resnet50

```
import torch
```

```
from torchvision import models
```

```
# Load pre-trained ResNet50 weights and transforms
```

```
weights = models.ResNet50_Weights.IMAGENET1K_V2
```

```
model = models.resnet50(weights=weights)
```

```
# Preprocess the image using the transformations from the imagenet dataset
```

```
preprocess = weights.transforms()
```

```
# Remove final Imagenet classification layer, keep 2048 dense vector
```

```
# Take time to unpack this syntax in the practical notebook
```

```
embedder = torch.nn.Sequential(*list(model.children())[:-1])
```

```
# Disable BatchNormalization
```

```
embedder.eval()
```

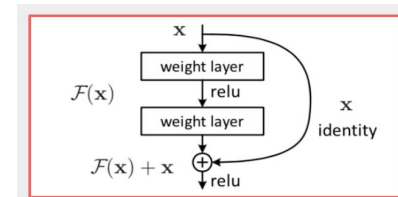
```
# Disable gradient computation
```

```
with torch.inference_mode():
```

```
    # unsqueeze(0) adds the batch size for inference
```

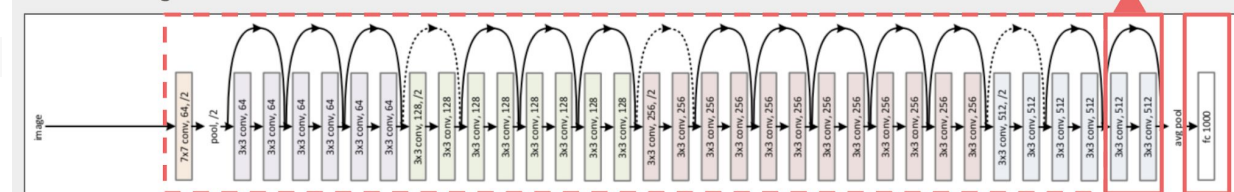
```
    # squeeze() removes the singleton dimensions from the output vector
```

```
    embedding = embedder(preprocess(image).unsqueeze(0)).squeeze()
```

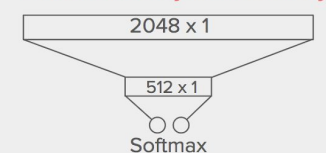


Residual Learning Block

ResNet50 Diagram



Re-architect fully-connected layers



CNN Features off-the-shelf: an Astounding Baseline for Recognition

Ali Sharif Razavian Hossein Azizpour Josephine Sullivan Stefan Carlsson
CVAP, KTH (Royal Institute of Technology)
Stockholm, Sweden

`{razavian, azizpour, sullivan, stefanc}@csc.kth.se`

https://www.csc.kth.se/~azizpour/papers/ha_cvpr14w.pdf

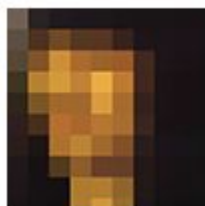
README.md

Art Recommendation system

This is the repository of a portfolio project at [DSR](#). This project aims to identify similar images using pre-trained computer vision networks. For an explanation of the technology see the [technology section](#).

Contributors

- [Catarina Ferreira](#)
- [Gargi Maheshwari](#)

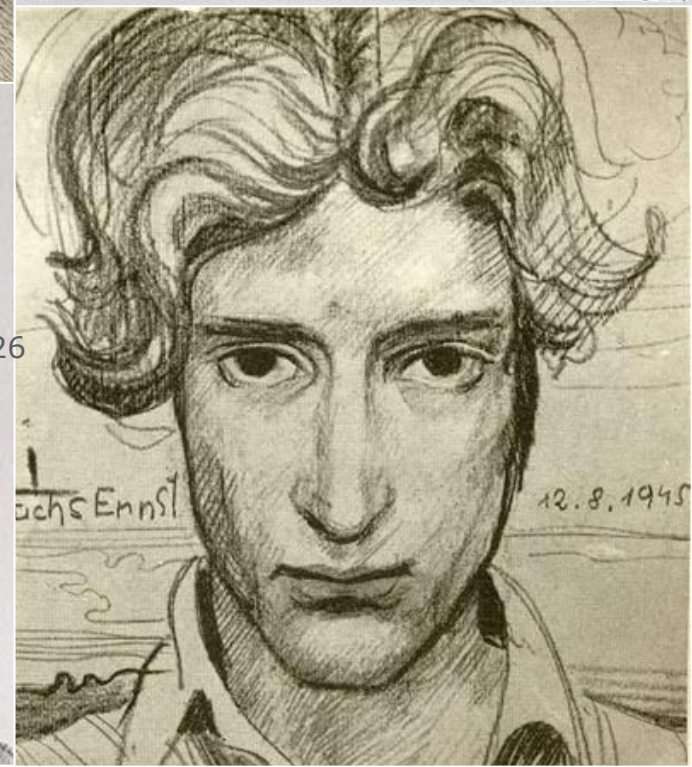
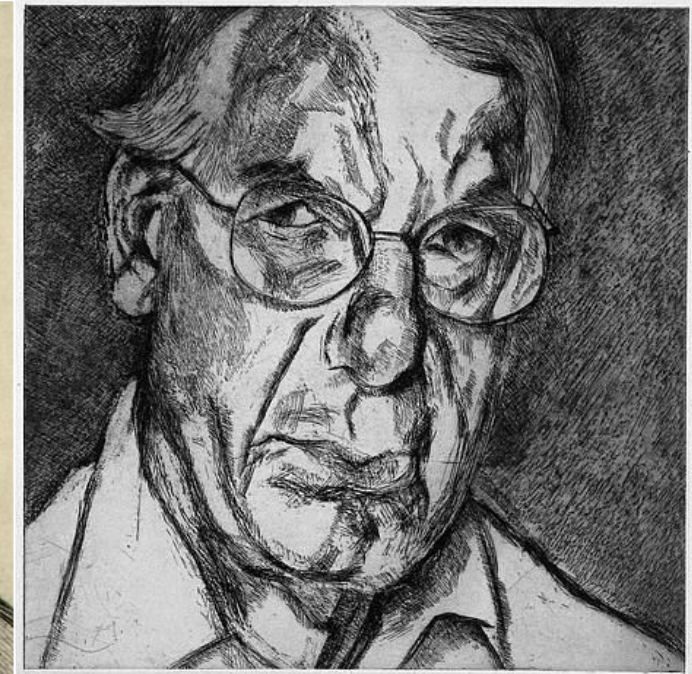


WIKIART
VISUAL ART ENCYCLOPEDIA

<https://github.com/gargimaheshwari/Wikiart-similar-art>





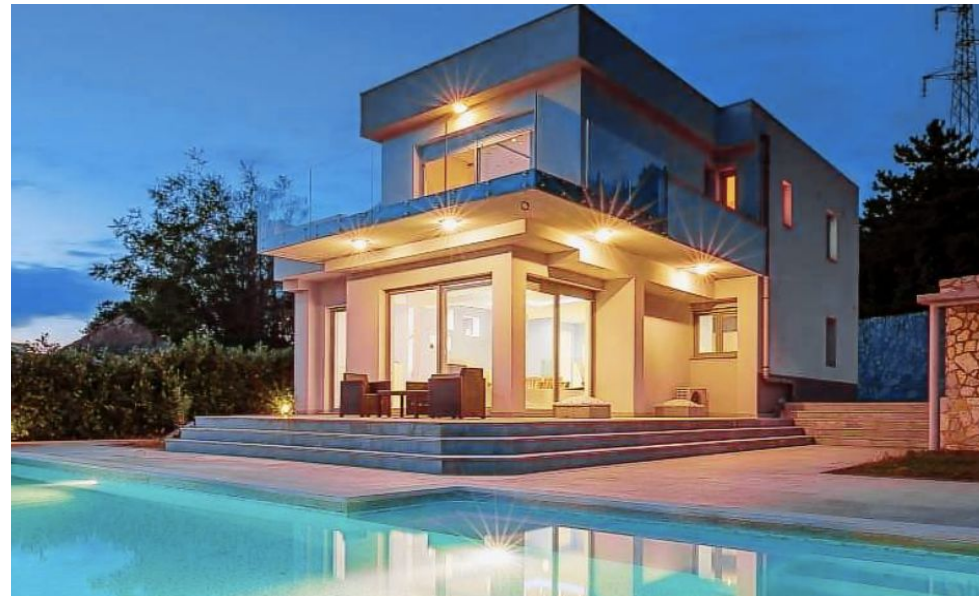
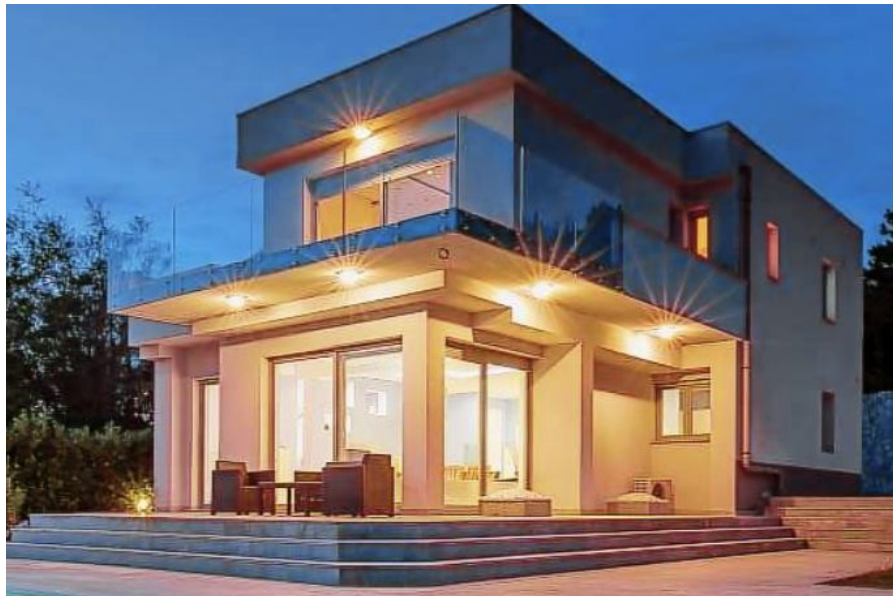


Neural embeddings as deduplicators



Cosine similarity = **0.45**

Neural embeddings as deduplicators



Cosine similarity = 0.89

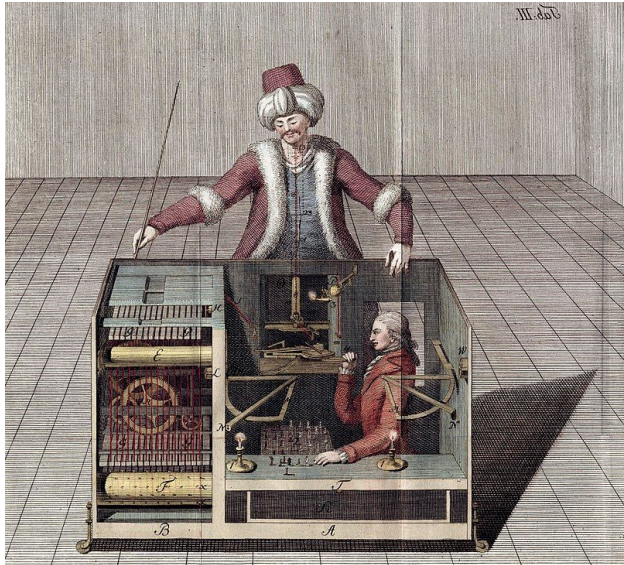
Benchmarks after thresholding on val set

Amazon Mechanical Turk

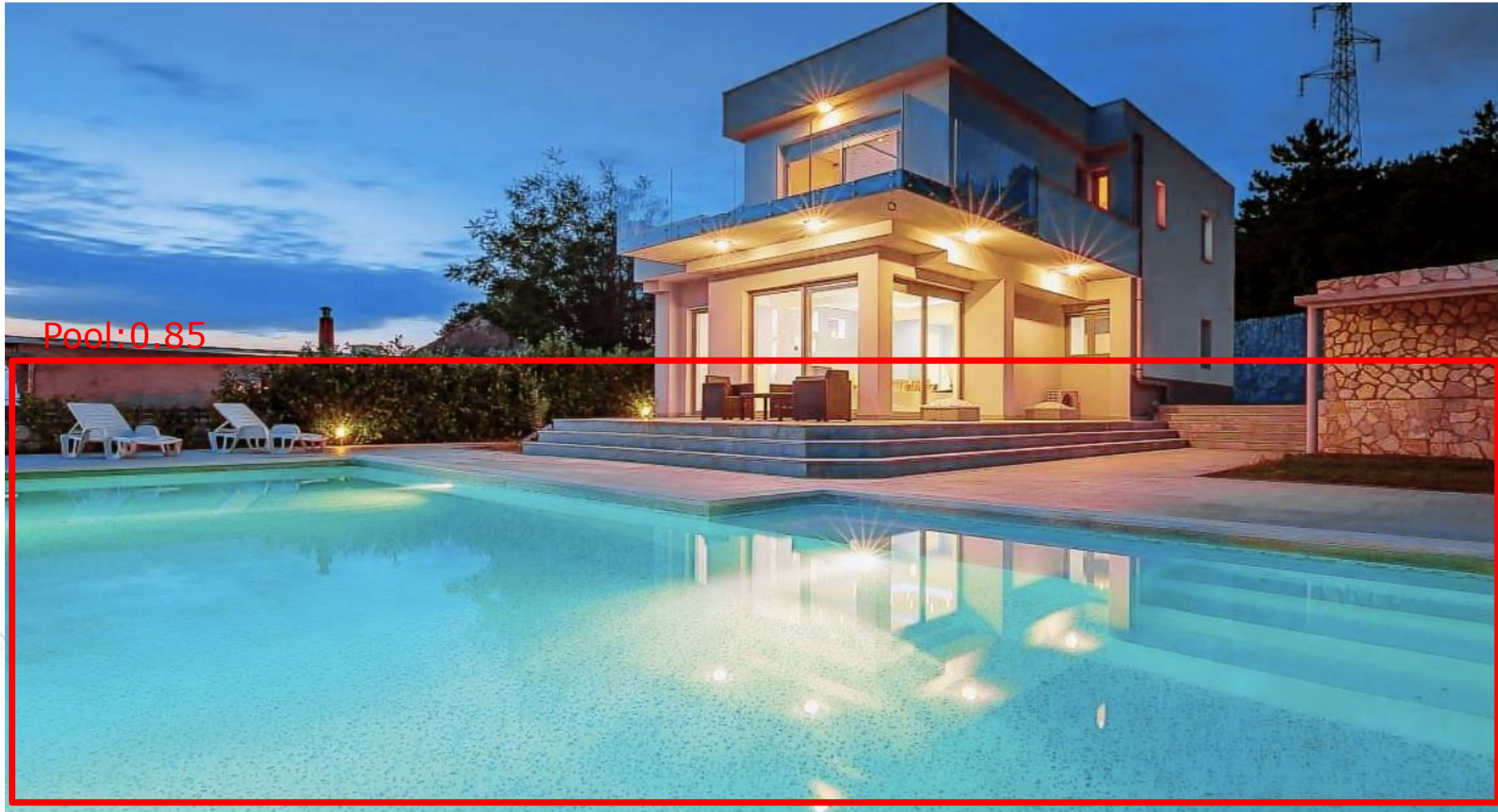
Access a global, on-demand, 24x7 workforce

Transformation Type	Precision	Recall
Crop	0.892	0.887
Crop & Zoom	0.885	0.891
Different Angle	0.753	0.748

<https://www.mturk.com/>

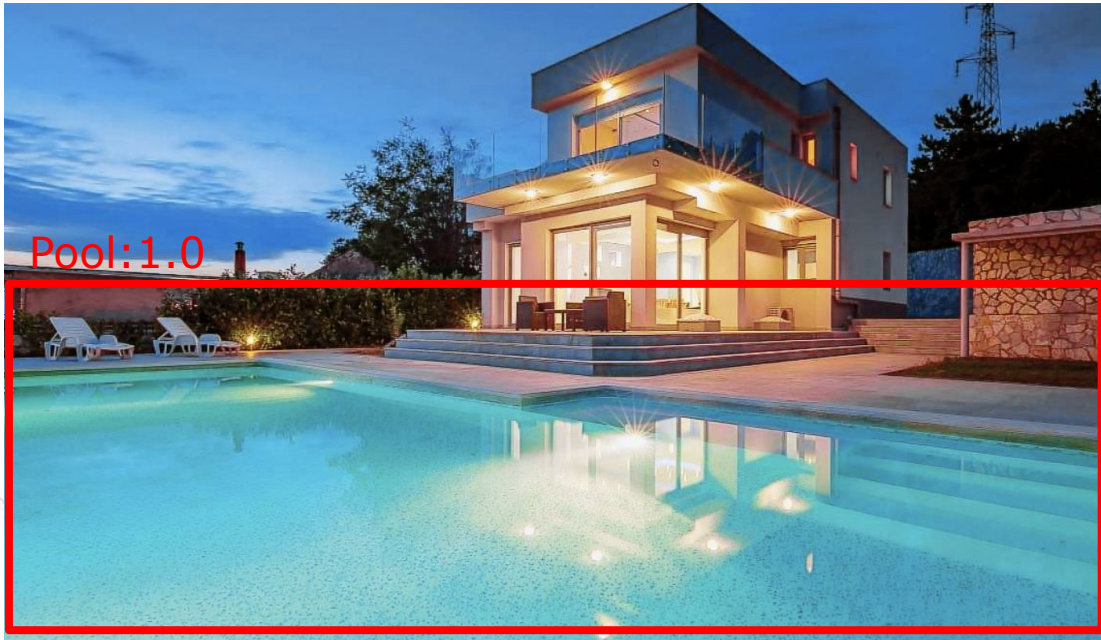


Round 2: image classification and object detection



Predicted class: 'facade', probability = 0.83

Inflated metrics and data leakage



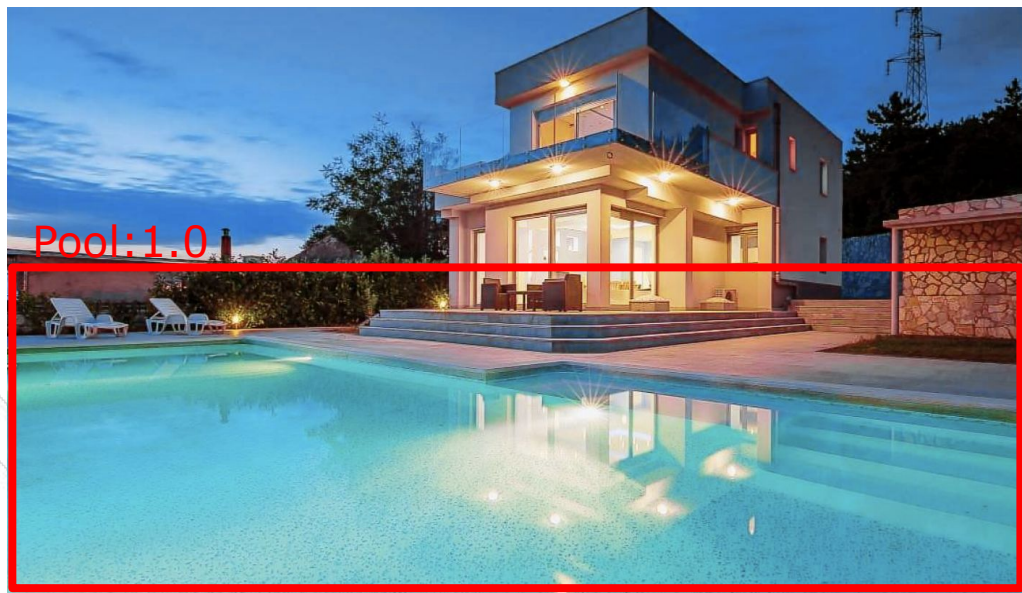
Predicted class: 'facade', probability = 0.99

Model	Precision	Recall
ACME's model	0.99	0.98
My model	0.86	0.84

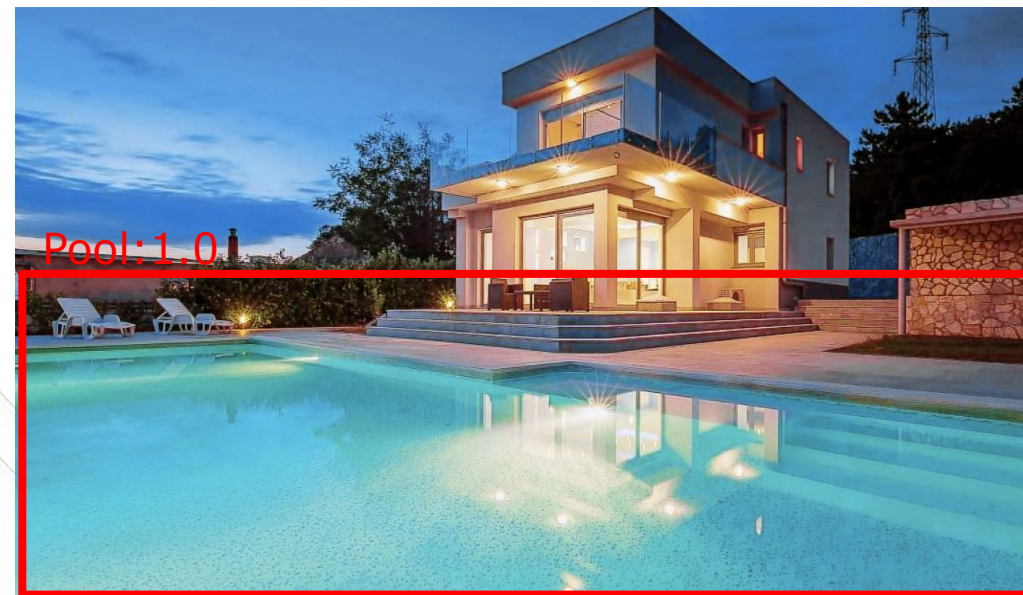
Model Performance Comparison for Facade Classification



Leaky splits



Train set
label = facade



Test set
label = facade

*Found about **20K** near and exact duplicates in the test set of **100K** images used by ACME*

Email to colleagues

Subject: Image Classification Accuracy Metrics - Data Leakage Issue

Hi all,

The image classification algorithm from **ACME** is reporting inaccurate metrics. The dataset isn't being deduplicated before train-validation-test splits. Same images appear across different splits, falsely inflating precision and recall scores.

To fix this, we need to:

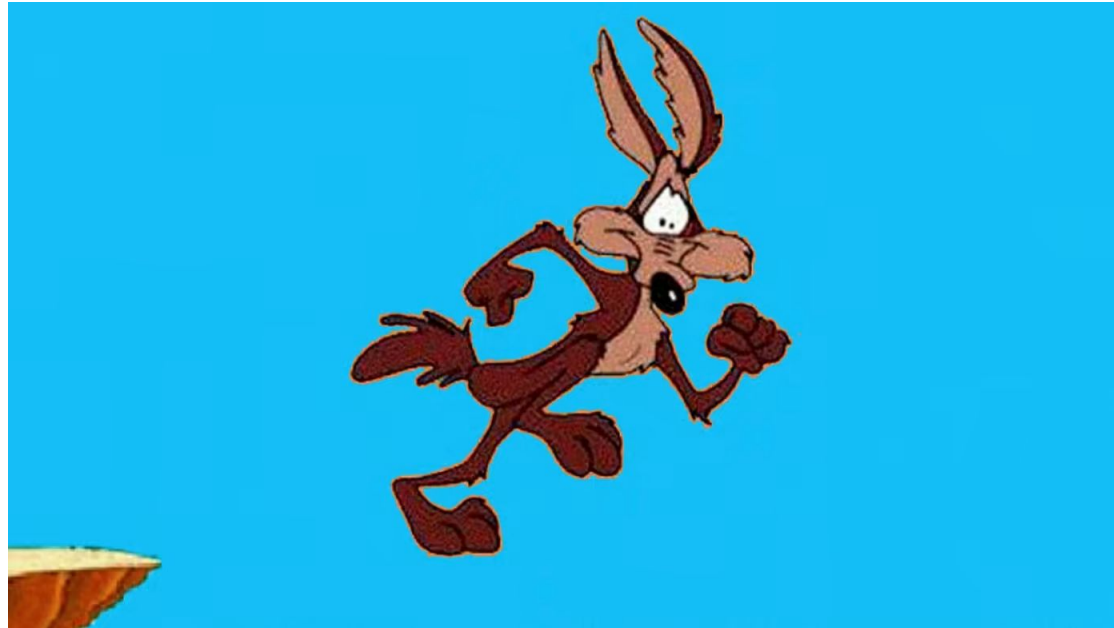
1. Deduplicate data before splitting
2. Re-run evaluation
3. Update performance metrics

Let me know if you'd like to discuss.

Best,

Antonio Rueda-Toicen

Data Scientist



Summary

Embeddings are a great way to deduplicate your model

- They outperform classical perceptual hashing methods in both precision and recall
- Cryptographic hashes only detect exact duplicates

Deduplicate your images before training a model

- Otherwise your performance metrics will be false and inflated

Further reading and references

Image deduplication in fiftyone

- https://docs.voxel51.com/recipes/image_deduplication.html
- <https://www.youtube.com/watch?v=AwWR8kWp8Bw>

On data leakage

- <https://voxel51.com/blog/on-leaky-datasets-and-a-clever-horse/>

Computing image uniqueness in fiftyone

- <https://docs.voxel51.com/tutorials/uniqueness.html>
- https://colab.research.google.com/github/voxel51/fiftyone/blob/v1.3.0/docs/source/tutorials/image_embeddings.ipynb

Building computer vision products at Hometogo GmbH

- <https://www.slideshare.net/slideshow/building-computer-vision-products-at-hometogo-gmbh-ai-in-action-berlin-april-2020/232995755>

Practical Computer Vision with PyTorch at OpenHPI


Channels
Courses
Podcast
News

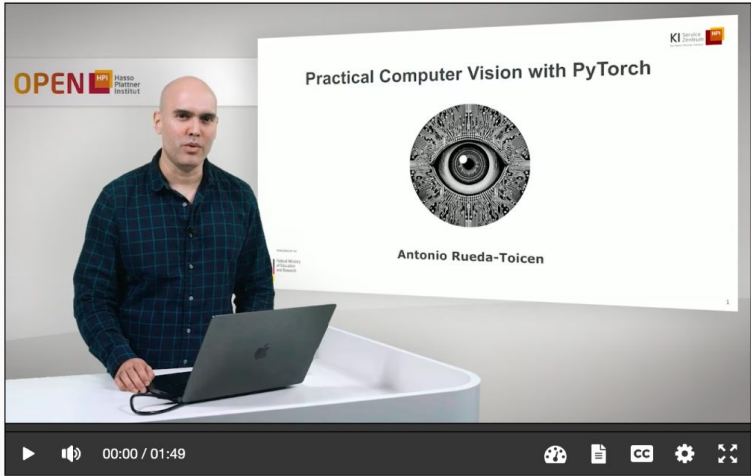
English
0 XP

Course has not yet started

Practical Computer Vision with PyTorch

Offered by Antonio Rueda-Toicen

Learnings
Discussions
Progress
Certificates
Course Details
Announcements



Practical Computer Vision in PyTorch is a comprehensive, hands-on course designed for developers and practitioners eager to explore computer vision using PyTorch. The course covers a wide spectrum of tasks, ranging from image classification and object detection to segmentation and generative modeling. With a strong emphasis on hands-on implementation, the participants will engage in coding demos and build projects using industry-standard tools and libraries. By the end of the course, the participants will be equipped with the skills to build, fine-tune, and deploy computer vision models for real-world applications.

May 7, 2025 - May 21, 2025
Language: English
Deutsch, English
Advanced, Big Data and AI, Data Science

Enroll me for this course

